ARQUITECTURA MONOLITICA

La arquitectura monolitica es un modelo de desarrollo el cual se efectua en capaz, esta arquitectura se basa en 4 capaz las cuales son: usuario, interfaz, modulos y resultado.
esta modelo se usa principalmente en proyectos pequeños ya que tiene una arquitectura muy simplificada.
su principal ventaja es tener un rendimiento muy efectivo, es decir muy rapido, y su principal desventaja es debido a asu gran despliegue que no soporta mucho codigo.

ARQUITECTURA EN CAPAZ

La arquitectura en capas es una forma de organizar un sistema de software dividiéndolo en diferentes niveles, donde cada capa tiene una responsabilidad específica.
ademas de esto, es usada comunmente para proyectos grandes.
Su principal ventaja es ser muy efectiva para trabajos en equipo.
Su desventajas son el tener mucha complejidad al iniciar el proyecto,otra de estas es que los cambios en una capa pueden afectar a otras si están demasiado acopladas

EXPOSICIONES

ARQUITECTURA MVC

Arquitectura MVC (Modelo - Vista - Controlador)
Esta arquitectura se basa en dividir una aplicación en 3 componentes principales, los cuales son:
Modelo: Se encarga de gestionar los datos y las reglas de negocio.
Vista: Se encarga de mostrar la información y la interfaz con la que interactúa el usuario.
Controlador: Se encarga de recibir las acciones del usuario y coordinar la comunicación entre el modelo y la vista.
Principal ventaja: Permite organizar el código, separar las responsabilidades y facilitar el mantenimiento y las modificaciones del sistema.
Principal desventaja: Puede aumentar la complejidad del desarrollo y generar más archivos y componentes, especialmente en proyectos pequeños.

ARQUITECTUA CLIENTE - SERVIDOR

Esta arquitectura se basa en que el cliente solicite un servicio y el servidor de respuesta de ello. esta arquitectura tiene como base principal el usar 3 tipos diferentes de capaz las cuales son: 2 capaz: Estas capaz se basan en cliente servidor.
3 capaz: Estas capaz se basasn en cliente modelo servidor
N Capaz: Estas capaz se basan en presentación, lógica, cache, datos y almacenamiento.
Su principal ventaja es que muchos clientes pueden interactuar con este sistema y una de sus principales desventajas es que se puede bloquear el seridor y lo que provoca esto es perdida de datos

ARQUITECTURA DE MICROSERVICIOS

Divide la aplicación en servicios pequeños e independientes, cada uno con una función específica. Se despliegan por separado y se comunican entre sí mediante APIs.

Patrones de diseño:

Composite: Permite tratar un objeto individual y un grupo de objetos de la misma forma, mediante una estructura de árbol.

Flyweight: Comparte objetos para ahorrar memoria en lugar de crear uno nuevo cada vez.

ARQUITECTURA ORIENTADA A SERVICIOS (SOA)

Organiza el sistema como servicios reutilizables que se comunican a través de un bus de servicios (ESB), usando protocolos estándar como SOAP o REST.

Patrones de diseño:

Observer: Define una relación uno a muchos: cuando un objeto cambia de estado, avisa automáticamente a los que están suscritos.

Mediator: Centraliza la comunicación entre objetos en un solo intermediario, para que no se conecten directamente entre sí.

ARQUITECTURA HEXAGONAL

También llamada de puertos y adaptadores. Deja la lógica de negocio en el centro, aislada, y la conecta con el exterior (base de datos, interfaz, APIs) mediante puertos y adaptadores.

Patrones de diseño:

Strategy: Define una familia de algoritmos intercambiables y permite elegir cuál usar en tiempo de ejecución.

State: Permite que un objeto cambie su comportamiento cuando cambia su estado interno.

ARQUITECTURA DIRIGIDA POR EVENTOS

Los componentes se comunican produciendo y consumiendo eventos de forma asíncrona, sin conocerse directamente. Usa productores, consumidores y un intermediario de eventos (broker).

Patrones de diseño:

Command: Convierte una petición en un objeto, permitiendo encolarla, registrarla o deshacerla.

Chain of Responsibility: Pasa una solicitud por una cadena de manejadores; cada uno decide si la procesa o la envía al siguiente.

ARQUITECTURA SERVERLESS

El código se ejecuta en funciones bajo demanda en la nube (FaaS), sin administrar servidores. El proveedor escala automáticamente y se paga solo por el uso.

Patrones de diseño:

Iterator: Permite recorrer los elementos de una colección sin exponer su estructura interna.

Memento: Guarda y restaura el estado anterior de un objeto sin violar su encapsulamiento.

Interpreter: Define una gramática para un lenguaje y un intérprete que evalúa sentencias de ese lenguaje.

ARQUITECTURA LIMPIA (CLEAN ARCHITECTURE)

Organiza el sistema en capas concéntricas (entidades, casos de uso, adaptadores e infraestructura) donde las dependencias siempre apuntan hacia adentro, hacia las reglas de negocio.

Patrones de diseño:

Template Method: Define el esqueleto de un algoritmo en una clase base y deja que las subclases redefinan algunos pasos.

Visitor: Permite añadir nuevas operaciones a una estructura de objetos sin modificar sus clases.